NLP Laboratory — Assignment 8

Programmatic Solutions to the Mid-Semester Questions and a Noisy-Channel Spell Corrector

Name	GAURAV DAMOR	**Course**	Natural Language Processing (AI357)
Roll No.	U24AI026	**Lab**	Lab 8 — Midsem Programs & Spell Correction

5 / 5 Midsem questions solved by code	**90.1%** Accuracy, 1-edit non-word errors	**29.2%** Overall accuracy, 1-edit model	**38.9%** Overall accuracy, 2-edit model

Summary of Answers

Question	Topic	Answer
Q1	FST, noun number	9-state transducer (Fig. A1, Table A1); +PL → "es" after s, x, z, ch, sh, else "s"
Q2	WordPiece	Merges: ##s+##t, l+##o; newest → n ##e ##w ##e ##st
Q3	TF-IDF	S1 and S3 are closest (cosine 0.8719)
Q4	Naive Bayes	E8 → NotSpam (1/112 vs 8/63)
Q5	Kneser-Ney	P(AI|love) = 3/8 = 0.3750; P(NLP|like) = 7/12 = 0.5833
Spell	Noisy channel	29.2% (1 edit) / 38.9% (2 edits) top-1 over all 39,507 pairs; 90.1% on single-edit non-word errors

Part A — Programmatic Solutions to the Mid-Semester Questions

Each question is solved by a short program in `src/q1_fst.py` … `src/q5_kneser_ney.py`. The programs print every intermediate quantity; the tables below are generated from their output (`output/midsem_solutions.json`), so the hand computation and the code are guaranteed to agree.

Q1. FST for English noun singular / plural forms

The transducer reads the **lexical tape** (stem letters followed by **+N** and **+SG** or **+PL**) and writes the **surface tape**. Letters are copied (l:l), +N and +SG are rewritten to ε, and +PL is rewritten to **s**, or to **es** when the stem ends in s, x, z, ch or sh. The stem states remember just enough of the stem's ending to make that decision: a final *c* or *s* may still become *ch* / *sh*, so they get their own states. Start state q0; accepting state q8.

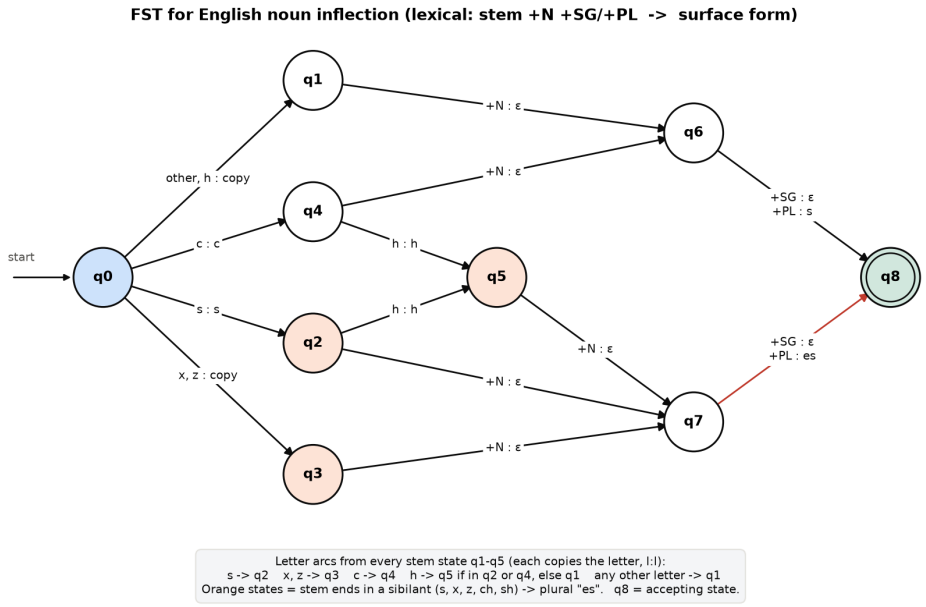


Figure A1: State diagram of the noun-inflection FST (input : output on each arc).

State	Meaning	s	x/z	c	h	other l	+N	+SG	+PL
q0	start	q2, s:s	q3, x/z:x/z	q4, c:c	q1, h:h	q1, l:l	—	—	—
q1	stem, last letter ordinary	q2, s:s	q3, x/z:x/z	q4, c:c	q1, h:h	q1, l:l	q6, +N:ε	—	—
q2	stem ends in 's'	q2, s:s	q3, x/z:x/z	q4, c:c	q5, h:h	q1, l:l	q7, +N:ε	—	—
q3	stem ends in 'x' or 'z'	q2, s:s	q3, x/z:x/z	q4, c:c	q1, h:h	q1, l:l	q7, +N:ε	—	—
q4	stem ends in 'c'	q2, s:s	q3, x/z:x/z	q4, c:c	q5, h:h	q1, l:l	q6, +N:ε	—	—
q5	stem ends in 'ch' or 'sh'	q2, s:s	q3, x/z:x/z	q4, c:c	q1, h:h	q1, l:l	q7, +N:ε	—	—
q6	+N read, regular stem	—	—	—	—	—	—	q8, +SG:ε	q8, +PL:s
q7	+N read, sibilant stem	—	—	—	—	—	—	q8, +SG:ε	q8, +PL:es
q8	final (accept)	—	—	—	—	—	—	—	—

Table A1: Complete transition table. Each cell is *next state, input:output*; — means no transition (the input is rejected).

Lexical stem	+N+SG	+N+PL		Lexical stem	+N+SG	+N+PL
cat	cat	cats		bench	bench	benches
dog	dog	dogs		dish	dish	dishes
book	book	books		brush	brush	brushes
fox	fox	foxes		wish	wish	wishes
box	box	boxes		month	month	months
bus	bus	buses		stomach	stomach	stomaches
glass	glass	glasses		epoch	epoch	epoches
buzz	buzz	buzzes		zinc	zinc	zincs
quiz	quiz	quizes		path	path	paths
church	church	churches		bath	bath	baths
watch	watch	watches				

Table A2: Forms generated by running the FST on test stems.

Running the same machine in reverse (surface → lexical) gives the analyses: **foxes** → fox+N+PL, foxe+N+PL, foxes+N+SG; **cats** → cat+N+PL, cats+N+SG; **churches** → church+N+PL, churche+N+PL, churches+N+SG; **dishes** → dish+N+PL, dishe+N+PL, dishes+N+SG; **fox** → fox+N+SG; **buses** → bus+N+PL, buse+N+PL, buses+N+SG; **months** → month+N+PL, months+N+SG; **stomachs** → stomachs+N+SG. Without a stem lexicon the inverted FST also proposes non-words such as *foxe+N+PL* or *cats+N+SG*. Composing it with a lexicon FSA of valid stems would remove them. The rule given in the question produces *quizes* and *stomaches*; the correct forms need two further rules (consonant doubling, and *ch* pronounced /k/), which are outside the stated specification.

Q2. WordPiece — two merges and the tokenisation of *newest*

Word frequencies: low: 5, lower: 2, lowest: 3, new: 4, newer: 2. Each word starts as its characters, with the first character unprefixed and the rest prefixed by ##. WordPiece merges the pair with the highest score freq(ab) / (freq(a) · freq(b)), which favours pairs whose parts rarely occur apart. BPE would instead take the most frequent pair.

Merge 1: ##s + ##t -> ##st

Pair	freq(ab)	freq(a)	freq(b)	score
##s + ##t	3	3	3	**0.3333**
l + ##o	10	10	10	0.1000
##e + ##r	4	13	4	0.0769
##e + ##s	3	13	3	0.0769
n + ##e	6	6	13	0.0769
##o + ##w	10	10	16	0.0625
##w + ##e	7	16	13	0.0337

Table A3: Top pair scores before merge 1 (all pairs are in the JSON output).

Merge 2: l + ##o -> lo

Pair	freq(ab)	freq(a)	freq(b)	score
l + ##o	10	10	10	**0.1000**
##e + ##r	4	13	4	0.0769
##e + ##st	3	13	3	0.0769
n + ##e	6	6	13	0.0769
##o + ##w	10	10	16	0.0625
##w + ##e	7	16	13	0.0337
##e + ##w	6	13	16	0.0288

Table A4: Top pair scores before merge 2 (all pairs are in the JSON output).

```
Vocabulary after two merges: { ##e, ##o, ##r, ##s, ##t, ##w, l, n, ##st, lo }. Tokenising
newest greedily, longest match first:
remaining "newest": longest vocabulary match = n
remaining "ewest": longest vocabulary match = ##e
remaining "west": longest vocabulary match = ##w
remaining "est": longest vocabulary match = ##e
remaining "st": longest vocabulary match = ##st
```

Answer: merges = **##s + ##t** → **##st** (score 1/3), then **l + ##o** → **lo** (score 0.1). *newest* → **n ##e ##w ##e ##st**.

Q3. TF-IDF sentence vectors and the closest pair

TF = raw count; IDF = $\log_{10}(N / df)$, unsmoothed, N = 4. Each sentence is a TF-IDF vector over the vocabulary, and closeness is measured by cosine similarity.

Term	df	idf	TF S1	TF S2	TF S3	TF S4	TF-IDF S1	TF-IDF S2	TF-IDF S3	TF-IDF S4
cat	3	0.1249	1	2	0	1	0.1249	0.2499	0.0000	0.1249
dog	3	0.1249	1	0	1	1	0.1249	0.0000	0.1249	0.1249
animal	2	0.3010	1	0	1	0	0.3010	0.0000	0.3010	0.0000
pet	3	0.1249	0	1	1	1	0.0000	0.1249	0.1249	0.1249
|v|							0.3491	0.2794	0.3491	0.2164

Table A5: Term statistics, TF and TF-IDF vectors (S1 = "cat dog animal", S2 = "cat cat pet", S3 = "dog pet animal", S4 = "cat dog pet").

Pair	v_a · v_b	|v_a|	|v_b|	cosine	Euclidean
S1-S2	0.0312	0.3491	0.2794	0.3201	0.3707
S1-S3	0.1062	0.3491	0.3491	**0.8719**	0.1767
S1-S4	0.0312	0.3491	0.2164	0.4133	0.3259
S2-S3	0.0156	0.2794	0.3491	0.1601	0.4107
S2-S4	0.0468	0.2794	0.2164	0.7746	0.1767
S3-S4	0.0312	0.3491	0.2164	0.4133	0.3259

Table A6: Pairwise similarities.

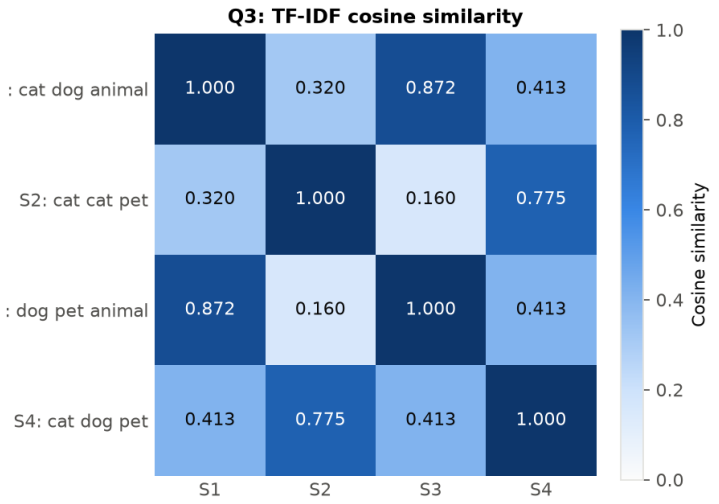


Figure A2: Cosine-similarity matrix of the four TF-IDF vectors.

Answer: S1 ("cat dog animal") and S3 ("dog pet animal") are closest, with cosine 0.8719. They share the rare, high-IDF term *animal* (idf 0.301), which dominates both vectors. Euclidean distance ties S1-S3 and S2-S4, because it is sensitive to vector length; cosine normalises length away and gives a unique answer. The choice of log base rescales every weight by the same factor, so the cosine ranking does not depend on it.

Q4. Naive Bayes spam classifier — E8 = (free = 0, win = 0, offer = 0)

Bernoulli Naive Bayes: $P(c \mid x) \propto P(c)\ \Pi_k\ P(x_k \mid c)$, where an absent word contributes $P(x_k = 0 \mid c) = 1 - P(x_k = 1 \mid c)$. The exam expects the unsmoothed MLE; the Laplace (add-1) version is shown as a check.

Estimate	Class	P(c)	P(free=0|c)	P(win=0|c)	P(offer=0|c)	P(c)·Π P(x|c)	Posterior
MLE	Spam	4/7	1/4	1/4	1/4	1/112 = 0.00893	0.0657
	NotSpam	3/7	2/3	2/3	2/3	**8/63 = 0.12698**	**0.9343**
Laplace	Spam	4/7	1/3	1/3	1/3	4/189 = 0.02116	0.1861
	NotSpam	3/7	3/5	3/5	3/5	**81/875 = 0.09257**	**0.8139**

Table A7: Naive Bayes computation for E8. Spam: free, win and offer each appear in 3 of 4 spam mails; NotSpam: each appears in 1 of 3.

Answer: $P(\text{Spam}) \cdot (1/4)^3 = 1/112 \approx 0.0089 < P(\text{NotSpam}) \cdot (2/3)^3 = 8/63 \approx 0.1270$, so E8 is classified as **NotSpam** (posterior 0.934). Laplace smoothing gives the same decision (NotSpam, posterior 0.814). This is intuitive: spam mails almost always contain at least one trigger word, and E8 contains none.

Q5. Kneser-Ney smoothing with d = 0.5

$P_{KN}(w \mid v) = \max(C(vw) - d, 0) / C(v) + \lambda(v) \cdot P_{cont}(w)$, with $\lambda(v) = d \cdot |\{w' : C(vw') > 0\}| / C(v)$ and $P_{cont}(w) = |\{v' : C(v'w) > 0\}|$ / (number of bigram types). $C(v)$ counts bigram tokens that begin with v; no boundary markers are used.

Bigram counts (8 types): I like: 2, like NLP: 2, like AI: 1, You like: 1, You love: 1, love AI: 1, We love: 1, love NLP: 1.

Query	C(vw)	C(v)	Discounted term	Followers of v	λ(v)	Preceders of w	P_cont(w)	P_KN
P_KN(AI | love)	1	2	1/4	AI, NLP	1/2	like, love	1/4	**3/8 = 0.3750**
P_KN(NLP | like)	2	3	1/2	AI, NLP	1/3	like, love	1/4	**7/12 = 0.5833**

Table A8: Kneser-Ney computation.

Answer: $P_{KN}(\text{AI} \mid \text{love}) = (1 - 0.5)/2 + (0.5 \cdot 2/2) \cdot (2/8) = 0.25 + 0.125 =$ **3/8 = 0.3750**.
$P_{KN}(\text{NLP} \mid \text{like}) = (2 - 0.5)/3 + (0.5 \cdot 2/3) \cdot (2/8) = 0.5 + 0.0833 =$ **7/12 = 0.5833**.

Part B — Noisy-Channel Spelling Correction

B1. Data

spell-errors.txt lists correct words with their observed misspellings (e.g. *raining: rainning, raning*; a suffix *n is a repeat count). After lowercasing (underscores become spaces), it yields **39,507** distinct (correct, misspelling) pairs covering 41,635 error tokens. 83 capitalisation-only "errors" are dropped. **count_unigram.txt** gives web frequencies for 333,333 words, and **count_1edit.txt** gives 1,587 single-edit types (39,073 occurrences). The true Damerau-Levenshtein distance between misspelling and correct word is 1 for 14,168 pairs, 2 for 9,782, 3 for 6,781 and 4 or more for 8,776. Only 35.9% are true single-edit errors. Also, 25.8% of the misspellings are themselves in the web vocabulary: real-word errors such as *fore* for *four*, and misspellings common enough on the web to be counted, such as *rainning* or *begining*.

Edit notation. An entry x | y means the writer typed x where the correct word has y, and > marks the start of the word. For example `t | te` means the e after t was dropped. This direction was verified, not assumed. Every single-insertion/deletion pair in spell-errors.txt was aligned, and the derived counts were correlated with count_1edit.txt: Pearson r = 0.88 under the typo|correct reading versus 0.72 under the reverse reading.

B2. Model

```
c* = argmax_c P(c) · P(x | c) over candidates c within k edits of the misspelling x
P(c) = count(c) / N (unigram language model, count_unigram.txt)
P(x | c) = p_err · Π_i P(e_i) (edits e_i that turn c into x)
joint: P(e) = count(e) / Σ_e' count(e') (Norvig)
conditional: P(e) = count(e) / count(correct side of e in corpus) (Kernighan, Church & Gale)
```

The conditional estimator is the textbook form, for example del[x,y] / count[xy]. Its denominators are the frequency-weighted counts of the 1- or 2-character string in the unigram corpus. Unseen edits get a pseudo-count of 0.5. Candidates are generated by a depth-first search over a trie of all vocabulary prefixes, which only extends prefixes of real words. That makes the distance-2 search cheap (about 8 ms per word over 333,333 words); all 36,913 unique misspellings are processed in parallel. When several edit sequences reach the same candidate, the most probable one is used.

Evaluation setting. Every input in spell-errors.txt is a known misspelling. The main setting therefore assumes **error detection is given** and never returns the input unchanged, which is the classic non-word correction task. In that setting p_err only scales every score equally and does not affect the ranking. Section B5 also reports the **open** setting, where the input itself is a candidate with P(x|x) = 1 – p_err. Accuracy is top-1 over all distinct pairs; token-weighted accuracy, which counts repeated errors, is also stored in the JSON output.

B3. Results

Scoring model	k = 1: accuracy	k = 1: token acc.	k = 2: accuracy	k = 2: token acc.
Prior only P(c)	26.12%	27.01%	25.63%	25.57%
Channel only P(x|c)	20.32%	20.63%	26.55%	26.60%
Noisy channel, joint P(edit)	28.30%	29.42%	**38.91%**	39.63%
Noisy channel, conditional P(edit)	**29.25%**	30.45%	37.11%	38.01%
Oracle: correct word among candidates	35.02%		59.39%	

Table B1: Top-1 accuracy over all 39,507 pairs (error detection given). k = maximum edit distance of candidates. The oracle row is the upper bound set by candidate generation.

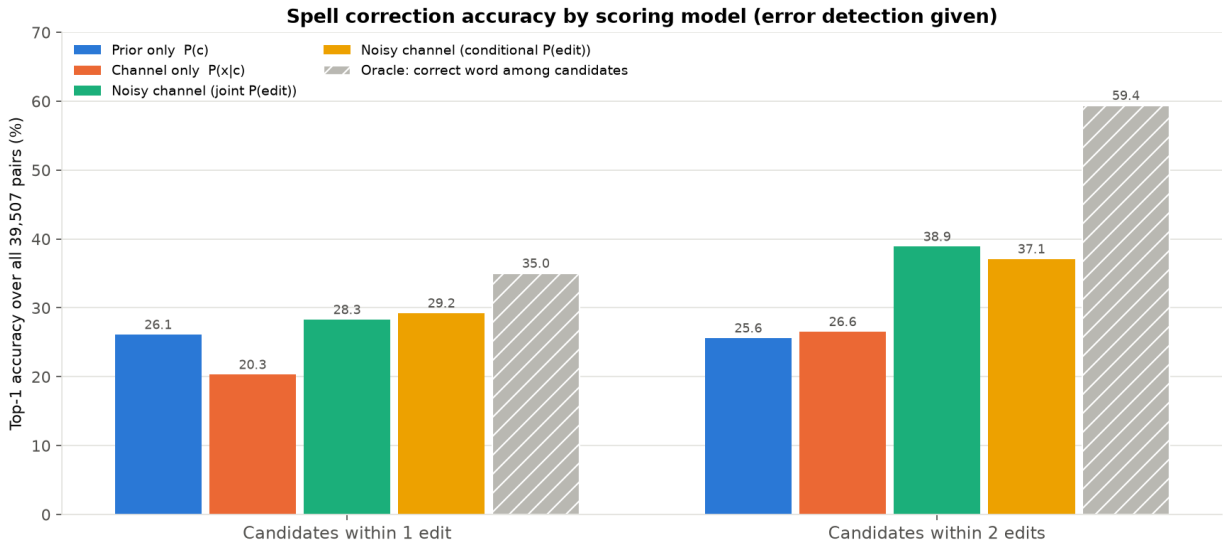


Figure B1: Accuracy of each scoring model against the candidate-generation oracle.

Subset	Pairs	k=1 joint	k=1 cond.	k=2 joint	k=2 cond.
distance=1 non-word (classic setting)	8,577	88.5%	90.1%	88.0%	90.1%
correct word reachable	varies	80.8%	83.5%	65.5%	62.5%
distance=1	14,168	78.9%	81.5%	78.5%	81.5%
distance=2	9,782	0.0%	0.0%	43.2%	31.6%
distance=3+	15,557	0.0%	0.0%	0.1%	0.1%
typo is a non-word	29,304	25.9%	26.4%	39.8%	36.8%
typo is a vocabulary word	10,203	35.2%	37.5%	36.3%	37.9%

Table B2: Accuracy by subset. "Correct word reachable" is the subset where the correct word is among the candidates (its size differs for k = 1 and k = 2).

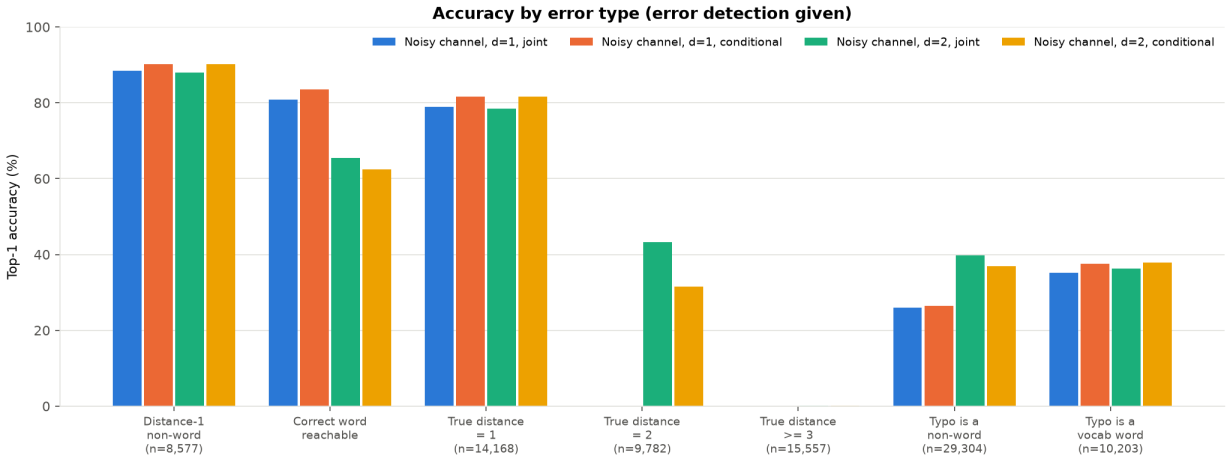


Figure B2: Accuracy by error type for the four noisy-channel configurations.

Configuration	Top-1	Top-3	Top-5
Noisy channel, k=1, joint	28.3%	32.9%	33.9%
Noisy channel, k=1, conditional	29.2%	33.4%	34.2%
Noisy channel, k=2, joint	38.9%	49.1%	51.8%
Noisy channel, k=2, conditional	37.1%	47.9%	50.6%

Table B3: Top-k accuracy (the correct word is among the k best-ranked suggestions).

B4. Leakage check: held-out channel counts

Every edit we can align in spell-errors.txt also appears in count_1edit.txt, so the channel model was built from the same errors it is tested on. To measure the effect, a 5-fold cross-validation grouped by correct word re-estimates the edit counts from the four training folds alone (aligning each pair of distance ≤ 2) and tests on the held-out fold.

Configuration	count_1edit.txt	Held-out counts	Difference
k=1, joint	28.32%	28.10%	-0.22 pts
k=1, conditional	29.27%	29.31%	+0.03 pts
k=2, joint	38.91%	38.48%	-0.43 pts
k=2, conditional	37.13%	37.19%	+0.06 pts

Table B4: 5-fold cross-validated accuracy (mean over folds).

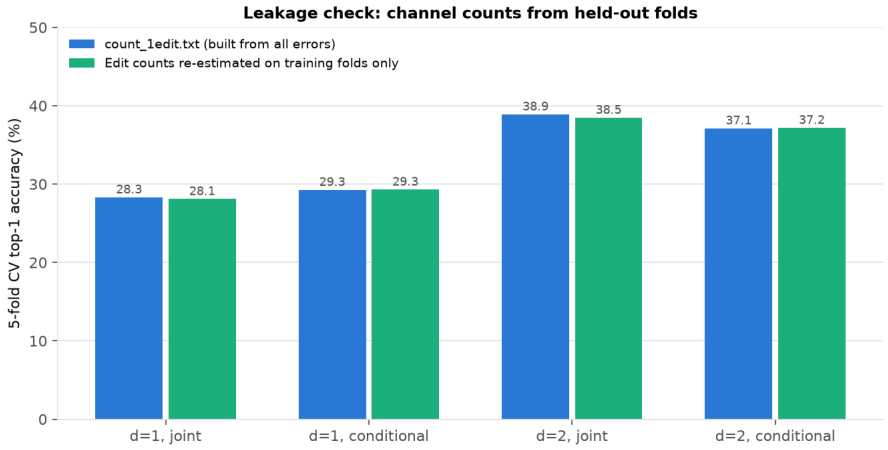


Figure B3: Channel counts from count_1edit.txt vs counts re-estimated on training folds only.

B5. Open setting: should the input itself be a candidate?

If the misspelling may also be returned unchanged, the 26% of misspellings that are web-vocabulary words compete as candidates. With p_err = 0.05 and the joint channel, accuracy falls from 28.3% to 19.9% (k = 1) and from 38.9% to 30.3% (k = 2). Raising p_err recovers accuracy steadily, reaching 27.2% at p_err = 0.99 (k = 1), because on this test set every input is an error. The conditional channel is flat across p_err. Its edit probabilities are counts of about 40k observed errors divided by character counts in a corpus of about 10^{11} tokens. They are correctly scaled *relative to each other*, but far smaller than any p_err could offset, so the unchanged input always wins when it is a vocabulary word.

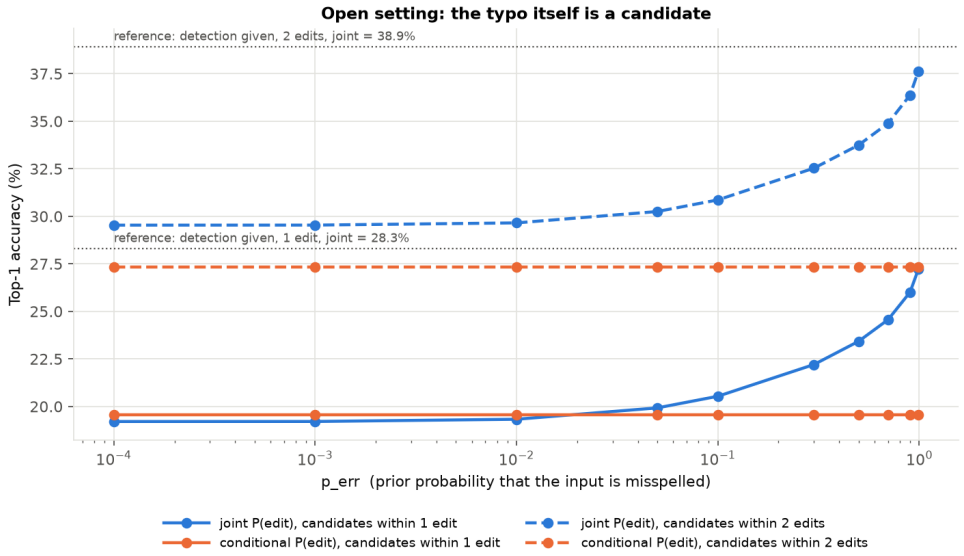


Figure B4: Open-setting accuracy as a function of p_err.

B6. Example corrections

Misspelling	Correct	Dist.	k=1 top-3 (joint)	k=2 prediction	k=1 result
pigen	pigeon	1	pigeon, pgen, igen	page	correct
exsibition	exhibition	1	exhibition, exibition	exhibition	correct
surplis	surplus	1	surplus, surpris	surplus	correct
neddle	needle	1	needle, meddle, neddie	middle	correct
counscious	conscious	1	conscious	conscious	correct
temporarrly	temporarily	1	temporarily	temporarily	correct
baragining	bargaining	1	bargaining	bargaining	correct
inconvenince	inconvenience	1	inconvenience	inconvenience	correct
coform	conform	1	conform	conform	correct
intertain	entertain	1	entertain	entertain	correct
cack	cake	2	cock, pack, sack	cock	wrong
a veod	avoid	3	(no candidate)	aviod	wrong
slow	so	2	low, flow, snow	low	wrong
tate	traffic	5	date, state, take	date	wrong
occaisionly	occasionally	3	(no candidate)	occaisionally	wrong
catologes	catalogues	2	(no candidate)	catalogs	wrong
fretenery	fraternally	5	(no candidate)	freeney	wrong
accewed	accrued	2	acceded	accessed	wrong
bearied	buried	2	bearded, berried, wearied	beard	wrong
snoraty	sorority	3	(no candidate)	snooty	wrong

Table B5: Randomly sampled successes and failures. For the example in the assignment, *rainning* → raining, trainning and *raning* → rating, ranging, raining (top-3, k = 1, joint).

B7. Discussion

Where the noisy channel works. On the classic task (single-edit, non-word errors) the corrector is right 90.1% of the time with the conditional channel (88.5% joint). Combining prior and channel clearly beats either alone: at k = 2 the prior alone gives 25.6%, the channel alone 26.6%, and the noisy channel 38.9%. The prior alone picks frequent words such as *the* or *a*; the channel alone picks the easiest edit regardless of whether the word is plausible.

Where it does not. Overall accuracy is bounded by candidate generation: only 35.0% of correct words lie within one edit, and 59.4% within two. The Birkbeck/Holbrook errors were written by poor spellers and are often phonetic (*negoshation*, *fretenery* for *fraternally*), far beyond two character edits. The remaining errors among reachable words are mostly real-word or context-dependent confusions (*cack* → *cock* vs *cake*, *fut* → *fat* vs *fruit*), which a unigram prior cannot resolve. Extending to k = 2 adds 10.6 points overall with almost no loss on single-edit errors, because two-edit candidates pay a second small edit probability. The joint estimator generalises better at k = 2, while the conditional estimator is slightly better at k = 1.

Leakage. Re-estimating the channel from held-out folds changes accuracy by at most 0.43 points. Common edits (vowel confusions, dropped doubled letters) recur across many different words, so the channel model learns general patterns rather than memorising specific test errors.

Next steps. (1) A bigram or trigram context model P(c | neighbours) to fix real-word errors. (2) Phonetic candidate generation (Soundex / Metaphone keys) for the distance-3+ errors. (3) Pruning web-noise entries from the vocabulary, or penalising them, so that an open-setting corrector can safely leave correctly spelled words alone.